



**WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA**
z siedzibą w Rzeszowie

Wstęp do programowania

Laboratorium nr 5

Ewelina Cupak, Nr albumu – 73132

Grupa laboratoryjna – L6

Informatyka, rok I, sem. I

Prowadzący:

Mgr Inż. Przemysław Skubel

Zadanie 1

Napisz funkcję, która oblicza pierwiastki funkcji kwadratowej. Funkcja przyjmuje argumenty a,b,c, które są współczynnikami funkcji kwadratowej. Dla a=0, napisz wyjątek, gdy funkcja nie jest kwadratowa. Skorzystaj z biblioteki math. Napisz dokumentację tej funkcji, kiedy jest co obliczane itp.

```
import math

def pierwiastki(a,b,c):

    """
    Funkcja oblicza pierwiastki kwadratowe równania  $ax^2 + bx + c = 0$ 

    Args:
    param a: współczynnik przy  $x^2$ , liczba rzeczywista. Nie może być 0.
    param b: współczynnik przy x, liczba rzeczywista.
    param c: wyraz wolny, liczba rzeczywista.

    returns:
    Funkcja zwraca krotkę z pierwiastkami równania kwadratowego.
    Jeśli delta < 0, zwraca informację o ujemnej delcie.
    Jeśli delta > 0, zwraca dwa pierwiastki równania.
    Jeśli delta = 0, zwraca dwa takie same pierwiastki.

    raises:
    Funkcja wyrzuca ValueError jeśli a = 0, bo nie jest to wtedy funkcja
    kwadratowa.
    """
    if a == 0:
        raise ValueError("a nie może być zerem")

    delta = b ** 2 - 4 * a * c

    if delta < 0:
        return "delta ujemna"

    if delta > 0:
        sqrtdelta = math.sqrt(delta)
        x1 = (-b + sqrtdelta) / (2*a)
        x2 = (-b - sqrtdelta) / (2*a)
        return round(x1, 1), round(x2, 1)
    else:
        return round(x1, 1), round (x1, 1)

print(pierwiastki(2,10,3))
```

Wynik programu:

```
C:\Users\ewel1\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\ewel1\PycharmProjects\PythonProject3\.venv\Zad. 1.py"
(-0.3, -4.7)
```

```
Process finished with exit code 0
```

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 1.py"
Traceback (most recent call last):
  File "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 1.py", line 41, in <module>
    print(pierwiastki(0,10,3))
  File "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 1.py", line 23, in pierwiastki
    raise ValueError("a nie może być zerem")
ValueError: a nie może być zerem

Process finished with exit code 1
```

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 1.py"
delta ujemna

Process finished with exit code 0
```

Zadanie 2

Napisz funkcję, która będzie tworzyć profil użytkownika z obowiązkowym imieniem oraz adresem email i dowolnymi dodatkowymi informacjami (min. 3). Funkcja zwraca słownik zawierający wszystkie dane profilu. Wykorzystaj funkcję `isinstance()` oraz `strip()`.

```
def funkcja(name: str, email: str, **kwargs):
    if not isinstance(name, str):
        raise ValueError("imie musi być stringiem")

    if not name.strip():
        raise ValueError("imie nie może być puste")

    if not isinstance(email, str):
        raise ValueError("email musi być stringiem")

    if not email.strip():
        raise ValueError("email nie może być pusty")

    if len(kwargs) < 3:
        raise ValueError("Musisz podać co najmniej 3 dodatkowe informacje")

    profile = {
        "name": name,
        "email": email
    }
    profile.update(kwargs)

    return profile

print(funkcja("Przemek", "ewelina@gmail", wiek = 20, adres = "Rzeszów",
ulubiony_kolor = "czerwony"))
```

Wynik programu:

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 2.py"
{'name': 'Przemek', 'email': 'ewelina@gmail', 'wiek': 20, 'adres': 'Rzeszów', 'ulubiony_kolor': 'czerwony'}

Process finished with exit code 0
```

Zadanie 3

Napisz funkcję, która zwraca listę tylko z liczbami parzystymi. Funkcja przyjmuje dowolną ilość argumentów. Wykorzystaj funkcję filter oraz lambda.

```
def parzyste(*args):  
    return list(filter(lambda x: x % 2 == 0, args))  
print(parzyste(1, 2, 3))  
print(parzyste(1, 2, 3, 4))
```

Wynik programu:

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 3.py"  
[2]  
[2, 4]  
  
Process finished with exit code 0
```

Zadanie 4

Napisz funkcję sum_digits(n), która oblicza sumę cyfr liczby rekurencyjnie. Napisz dokumentację tej funkcji, która opisywać będzie jej działanie.

```
def sum_digits(n):  
  
    """  
    Funkcja obliczająca rekurencyjnie sumę cyfr liczby.  
  
    param n: liczba całkowita, której cyfry zostaną zsumowane. Jeśli liczba  
    będzie ujemna, zostanie potraktowana jako dodatnia.  
  
    returns:  
    Zwraca, liczbę całkowitą, sumę cyfr liczby n.  
    """  
  
    n = abs(n)  
  
    if n < 10:  
        return n  
    else:  
        return (n % 10) + sum_digits(n // 10)  
  
print(sum_digits(-123))
```

Wynik programu:

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 4.py"  
6  
  
Process finished with exit code 0
```

Zadanie 5

Napisz funkcję zmieniającą miejscami dwa elementy listy wraz z obsługą wyjątków.

Wymagania

1. Funkcja przyjmuje listę oraz indeksy.
2. Działa in-place (modyfikacja istniejącej listy)
3. Obsługuje ujemne indeksy

4. Wyrzuca:

- a. ValueError, gdy lista jest pusta
- b. TypeError, gdy argumenty mają niewłaściwe typy
- c. IndexError, gdy indeksy wychodzą poza zakres

```
def zamien_miejscami(lista_docelowa: list, indeks1: int, indeks2: int):  
    if not isinstance(lista_docelowa, list) or not isinstance(indeks1, int)  
    or not isinstance(indeks2, int):  
        raise TypeError('lista_docelowa powinna być listą, indeksy powinny  
być liczbami całkowitymi')  
    if len(lista_docelowa) == 0:  
        raise ValueError("lista docelowa must be non-empty")  
    if not (0 <= indeks1 < len(lista_docelowa)) or not (0 <= indeks2 <  
len(lista_docelowa)):  
        raise IndexError("Indeksy poza zakresem listy")  
  
    if indeks1 < 0:  
        indeks1 += len(lista_docelowa)  
    if indeks2 < 0:  
        indeks2 += len(lista_docelowa)  
  
    lista_docelowa[indeks1], lista_docelowa[indeks2] =  
lista_docelowa[indeks2], lista_docelowa[indeks1]  
  
lista_docelowa = [1,2,3,4,5,6,7]  
print(lista_docelowa)  
zamien_miejscami(lista_docelowa, 1, 2)  
print(lista_docelowa)
```

Wynik programu:

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py"  
[1, 2, 3, 4, 5, 6, 7]  
[1, 3, 2, 4, 5, 6, 7]  
  
Process finished with exit code 0
```

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py"  
Traceback (most recent call last):  
  File "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py", line 17, in <module>  
    zamien_miejscami(lista_docelowa, "1", 2)  
  File "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py", line 3, in zamien_miejscami  
    raise TypeError('lista_docelowa powinna być listą, indeksy powinny być liczbami całkowitymi')  
TypeError: lista_docelowa powinna być listą, indeksy powinny być liczbami całkowitymi
```

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py"  
Traceback (most recent call last):  
  File "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py", line 17, in <module>  
    zamien_miejscami(lista_docelowa, 1, 2)  
  File "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py", line 5, in zamien_miejscami  
    raise ValueError("lista docelowa must be non-empty")  
ValueError: lista docelowa must be non-empty  
  
Process finished with exit code 1
```

```
C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Scripts\python.exe "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py"  
Traceback (most recent call last):  
  File "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py", line 17, in <module>  
    zamien_miejscami(lista_docelowa, 12, 2)  
  File "C:\Users\eweli\PycharmProjects\PythonProject3\.venv\Zad. 5.py", line 7, in zamien_miejscami  
    raise IndexError("Indeksy poza zakresem listy")  
IndexError: Indeksy poza zakresem listy  
  
Process finished with exit code 1
```

Zadanie 6

Odpowiedz na pytania i krótko uzasadnij odpowiedź:

1. Co się stanie? Czy kod się wykona, czy wystąpi błąd? Jeśli błąd, podaj jego typ i powód.

```
def t(x, *, y, z=0):
```

```
    return x + y + z
```

```
val= t(1, 2, z=3)
```

```
# Co się stanie? Czy kod się wykona, czy wystąpi błąd? Jeśli błąd, podaj
jego typ i powód.
# def t(x, *, y, z=0):
#     return x + y + z
# val= t(1, 2, z=3)

# Funkcja nie wykona się, ponieważ po * wszystkie argumenty funkcji powinny
być podane z nazwą.
# Próba wywołania funkcji z parametrami t(1,2,=3) nie uda się, ponieważ
parametr y jest wywołany pozycyjnie, a nie z nazwą.
# Wystąpi błąd typu TypeError.

#Poprawna funkcja
def t(x, *, y, z=0):
    return x + y + z
val= t(1, y=2, z=3)
print(val)
```

2. Wyjaśnij co dokładnie zwróci y oraz jak Python rozdzielił argumenty.

```
f = lambda a, /, *args, b=0: (a, sum(args) + b)
```

```
y = f(3, 4, 5, b=2)
```

```
# Wyjaśnij co dokładnie zwróci y oraz jak Python rozdzielił argumenty.
# f = lambda a, /, *args, b=0: (a, sum(args) + b)
# y = f(3, 4, 5, b=2)

# y zwróci krotkę z pierwszym elementem a = 3, oraz kolejnym sum(args) + b
= 4+5+2
# Rozdzielenie argumentów: argument a przed / musi być argumentem
pozycyjnym, kolejne argumenty są przedstawione jako krotka
# i trafiają do *args, argument b może ustawić jedynie z nazwą.
```

Wnioski ogólne: Zrealizowanym celem laboratoriów było zapoznanie się z funkcjami w języku Python. Na laboratoriach zajęliśmy tworzeniem funkcji oraz ich wywoływaniem, wyrzucaniem wyjątków, a także innymi operacjami związanymi z funkcjami. Kolejnym zadaniem było poznanie skróconych funkcji (lambda), odwołanie w funkcji do niej samej, a także dokładne poznanie parametrów w funkcjach i reguł z nimi związanych.